

Project FILM FOUND



Requirements for Project

1. Programming Language:

Python: Python is widely used in the field of data science and machine learning due to its extensive libraries and ease of use.

2. Data Manipulation and Analysis:

NumPy: A fundamental library for numerical computations in Python.

Pandas: A powerful library for data manipulation and analysis, which is particularly useful for handling tabular data.

3. Machine Learning Libraries:

Scikit-learn: A popular machine learning library that provides a variety of algorithms and tools for regression and classification tasks.

TensorFlow or PyTorch: Deep learning frameworks that enable the creation of complex neural network models for more advanced calorie burn prediction.

4. Data Visualization:

Matplotlib: A versatile library for creating visualizations and plots in Python.

Seaborn: Built on top of Matplotlib, Seaborn provides attractive and informative statistical graphics.

5. Model Selection and Evaluation:

Scikit-learn (again): Offers utilities for model selection through cross-validation, hyperparameter tuning, and evaluation metrics.

6. Data Preprocessing:

Scikit-learn: Provides various preprocessing modules for handling missing data, scaling features, and encoding categorical variables.

7. Feature Scaling:

Techniques like Min-Max scaling or Standardization might be required to normalize the data for better model performance.

8. Additional Libraries:

Pickle: For saving and loading trained machine learning models.

9. Jupyter Notebook :

For creating interactive and shareable data analysis and model development notebooks.

It's important to note that the choice of tools may vary depending on the specific project requirements and personal preferences. Python-based libraries and frameworks are generally preferred due to their robustness, extensive community support, and active development, making them suitable for developing and deploying machine learning models efficiently.

2.1 Introduction to project

PROJECT NAME- MOVIE_RECOMMENDATION_SYSTEM

A Movie Recommendation System is a tool that helps users discover films the user might enjoy based on their preferences, behavior, or characteristics of movies. It leverages various algorithms and data sources to suggest movies tailored to individual tastes. It combines data collection, algorithmic processing, and user interfaces to create a personalized viewing experience, enhancing user satisfaction and engagement. As technology evolves, these systems continue to refine their algorithms and approaches, striving to offer even more accurate and personalized movie suggestions.

2.2 Key Objectives:

- **Data Collection:** Gather data from sources like MovieLens or IMDB, including user ratings and movie details.
- **Data Preprocessing:** Clean and prepare the data for analysis, handling missing values and normalizing ratings.
- **Algorithm Implementation:** Develop and train recommendation models using techniques like matrix factorization or machine learning algorithms.
- **Recommendation Generation:** Use the trained model to generate and present movie recommendations based on user inputs or historical data.

Code to Project:

1. Import Libraries

```
# Import Libraries
import pandas as pd
import numpy as np
import difflib
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

The above libraries are imported to handle data manipulation (`pandas`, `numpy`), perform text vectorization (`TfidfVectorizer`), and measure similarity between movies (`cosine_similarity`).

2. Load Dataset

```
# Loading the data from the csv file to a pandas dataframe
movies_data = pd.read_csv('content/movies.csv')
```

```
# Printing the first 5 rows of the dataframe
movies_data.head()
```

	index	budget	genres	homepage	id	keywords	original_language	original_title	overview	popularity	runtime	spoken_languages
0	0	237000000	Action Adventure Fantasy Science Fiction	http://www.avatarmovie.com/	19995	future class future space war space colony 90...	en	Avatar	In the 22nd century, a paraplegic Marine is d...	180.437977	162.0	[[{"iso_639_1": "en", "name": "English"}, {"iso...
1	1	300000000	Adventure Fantasy Action	http://disney.go.com/disneypictures/pirates/	285	ocean drug abuse exotic island east India 1945...	en	Pirates of the Caribbean: At World's End	Captain Barbossa, long believed to be dead, ha...	130.082815	165.0	[[{"iso_639_1": "en", "name": "English"}]]
2	2	240000000	Action Adventure Crime	http://www.sonypictures.com/movies/spectre/	209647	spy based on novel secret agent action	en	Spectre	A cryptic message from Bond's past	107.376788	148.0	[[{"iso_639_1": "fr", "name": "Fran\u00e7ais"}],...

```
# Find the no. of rows and columns in our dataset
movies_data.shape
```

```
(4803, 24)
```

The dataset contains around 4800 movies with multiple attributes such as *title*, *genre*, *overview*, *popularity*, and *keywords*.

3. Select Features

```
# Selecting the relevant features for recommendation ( feature selection )
selected_features = ['genres', 'keywords', 'tagline', 'cast', 'director']
print(selected_features)
```

```
['genres', 'keywords', 'tagline', 'cast', 'director']
```

These features help describe the content of a movie and are used to calculate the similarity between different movies.

4. Handle Missing Values

```
# Find the Null values in the 'selected_features'
for feature in selected_features:
    print(feature, movies_data[feature].isnull().sum())
```

```
genres 28
keywords 412
tagline 844
cast 43
director 30
```

```
# Replacing the NULL values with Null string
for feature in selected_features:
    movies_data[feature] = movies_data[feature].fillna('')
```

Missing values in these columns are replaced with empty strings to avoid errors during text processing.

5. Combine Selected Features

```
# Combining all the 5 selected features
combined_features = movies_data['genres']+' '+movies_data['keywords']+' '+movies_data['tagline']+' '+movies_data['cast']+' '+movies_data['director']

print(combined_features)

0      Action Adventure Fantasy Science Fiction cultu...
1      Adventure Fantasy Action ocean drug abuse exot...
2      Action Adventure Crime spy based on novel secr...
3      Action Crime Drama Thriller dc comics crime fi...
4      Action Adventure Science Fiction based on nove...
...
4798     Action Crime Thriller united states\u2013mexic...
4799     Comedy Romance A newlywed couple's honeymoon ...
4800     Comedy Drama Romance TV Movie Date love at fir...
4801           A New Yorker in Shanghai Daniel Henney Eliza...
4802     Documentary obsession camcorder crush dream gl...
length: 4803, dtype: object
```

A single text column is created by merging all the selected features. This allows us to represent each movie as a single piece of text data.

6. Convert Text Data into Feature Vectors

```
# converting the text data to features vectors
vectorizer = TfidfVectorizer()
```

```
feature_vectors = vectorizer.fit_transform(combined_features)
```

The TF-IDF Vectorizer converts text into numerical vectors, representing the importance of words.

7. Compute Similarity Between Movies

```
from warnings import simplefilter
# Getting the Similarity score using Cosine Similarity
similarity = cosine_similarity(feature_vectors)
```

```
print(similarity)
```

```
[[1.          0.07219487 0.037733   ... 0.          0.          0.          ]
 [0.07219487 1.          0.03281499 ... 0.03575545 0.          0.          ]
 [0.037733   0.03281499 1.          ... 0.          0.05389661 0.          ]
 ...
 [0.          0.03575545 0.          ... 1.          0.          0.02651502]
 [0.          0.          0.05389661 ... 0.          1.          0.          ]
 [0.          0.          0.          ... 0.02651502 0.          1.          ]]
```

Cosine similarity calculates how close two movies are based on their text vectors. A value closer to 1 means the movies are more similar.

8. Get Movie Name from User

```
# Getting the movie name from the user
movie_name = input('Enter your favourite movie name: ')
```

```
Enter your favourite movie name: Iron Man
```

9. Find Closest Match

```
# Creating a list with all the movie name given in the dataset
list_of_all_titles = movies_data['title'].tolist()

# Find the close movie name given by the user
find_close_match = difflib.get_close_matches(movie_name, list_of_all_titles)

# Get the first movie
close_match = find_close_match[0]
```

`difflib.get_close_matches()` is used to handle spelling errors and find the closest title match in the dataset.

10. Get Index of Movie and Find Similar Movies

```
# Find the index of the movie and title
index_of_the_movie = movies_data[movies_data.title == close_match]['index'].values[0]

# Getting a list of similar movies
similarity_score = list(enumerate(similarity[index_of_the_movie]))

# Sorting the movies based on more similarity score
sorted_similar_movies = sorted(similarity_score, key = lambda x:x[1], reverse = True)
```

The selected movie's index is found, and the similarity scores are sorted in descending order to get the most similar movies.

11. Display Top 10 Recommended Movies

```
# Print the name of similar movies based on the index
print('Movies suggested for you: \n')

i = 1
for movie in sorted_similar_movies:
    index = movie[0]
    title_from_index = movies_data[movies_data.index == index]['title'].values[0]
    if (i < 10):
        print(i, '.', title_from_index)
        i += 1
```

Movies suggested for you:

```
1 . Iron Man
2 . Iron Man 2
3 . Iron Man 3
4 . Avengers: Age of Ultron
5 . The Avengers
6 . Captain America: Civil War
7 . Captain America: The Winter Soldier
8 . Ant-Man
9 . X-Men
```

The system recommends the top 10 movies similar to the one entered by the user based on cosine similarity of textual features.

12. Save Model

```
# Importing the pickle module to save Python objects to disk
import pickle
```

```
# Save the movies data (converted to dictionary) into a pickle file
pickle.dump(movies_data.to_dict(), open('movies_dict.pkl', 'wb'))

# Save the similarity matrix into another pickle file
pickle.dump(similarity, open('similarity.pkl', 'wb'))
```

13. User-Friendly Interface:

```
def fetch_poster(movie_id):  
    url = f"https://api.themoviedb.org/3/movie/{movie_id}?api_key=0f731ca6b61f1fac7d9c53aa31b7544"   
  
    session = requests.Session()  
    retry = Retry(connect=3, backoff_factor=1)  
    adapter = HTTPAdapter(max_retries=retry)  
    session.mount('http://', adapter)  
    session.mount('https://', adapter)  
  
    try:  
        response = session.get(url, timeout=5)  
        response.raise_for_status()  
        data = response.json()  
        return "https://image.tmdb.org/t/p/w500/" + data['poster_path']  
    except requests.exceptions.RequestException as e:  
        print(f"Error fetching poster for movie ID {movie_id}: {e}")  
        return "default_poster_url_or_path_here"  
  
def recommend(movie):  
    movie_index = movies[movies['title'] == movie].index[0]  
    distances = similarity[movie_index]  
    similar_movies = sorted(list(enumerate(distances)), reverse=True, key=lambda x: x[1])[1:6]  
  
    recommended_movies = []  
    recommended_movies_posters = []  
    for i in similar_movies:  
        movie_id = movies.iloc[i[0]].id  
        ## fetch poster from api  
  
        recommended_movies.append(movies.iloc[i[0]].title)  
        recommended_movies_posters.append(fetch_poster(movie_id))  
    return recommended_movies, recommended_movies_posters
```

Our report will propose a clean UI, allowing individuals to input their exercise details and receive real-time movie recommendation.

2.3 Model Used In Project

Content Based Filtering Model

Feature Extraction:

- **Tags Creation:** Combine different features of the movies, such as the overview, genres, keywords, cast, and crew, into a unified "tags" field. This allows the system to represent each movie with a comprehensive set of descriptive attributes.

TextVectorization:

- **Count Vectorizer:** Use CountVectorizer from Scikit-Learn to convert the text data into numerical format. This involves creating a matrix of token counts from the movie tags.

Similarity Calculation:

- **Cosine Similarity:** Compute the similarity between movies based on the vectorized tags using cosine similarity. This metric measures the cosine of the angle between two vectors, which in this context represents how similar two movies are based on their tags.

Recommendation Function:

- **Movie Recommendation:** Define a function to recommend movies based on the similarity scores. This function identifies the most similar movies to a given movie and retrieves their details.

2.4 Significance of Project

A movie recommendation system project is significant for enhancing user experience by providing personalized content suggestions, which boosts satisfaction and engagement. It utilizes user data, showcasing skills in data analysis and extraction of meaningful insights from large datasets—key components of data science and machine learning. The project also involves developing various algorithms, such as collaborative filtering and content-based filtering, fostering a deeper understanding of these techniques. In a competitive digital landscape, effective recommendations are crucial for user retention and loyalty, making this project highly relevant to market needs. Additionally, it combines elements of computer science, statistics, psychology, and design, promoting interdisciplinary learning. Simulating real-world challenges faced by companies like Netflix and Spotify, it offers practical experience applicable across industries. Moreover, there is potential for innovation, such as incorporating social factors or mood-based suggestions, allowing for creative problem-solving. Ultimately, this project not only enhances individual interactions with media but also contributes to cultural enrichment and community engagement.

2.5 Existing Problem

While movie recommendation systems offer significant benefits, several challenges and limitations can affect their effectiveness and user experience. Here are some common issues:

Cold Start Problem:

- **Challenge:** New movies or users have limited data, which makes it difficult to generate accurate recommendations.
- **Impact:** New content might not be recommended effectively, leading to a less engaging experience for users interested in the latest releases.

Data Sparsity:

- **Challenge:** User interaction data, such as ratings or reviews, can be sparse, especially for less popular movies.
- **Impact:** Sparse data can lead to less accurate recommendations, as the system lacks sufficient information to understand user preferences.

Scalability Issues:

- **Challenge:** As the number of movies and users increases, the computational complexity of generating recommendations can become a bottleneck.
- **Impact:** Larger datasets require more processing power and memory, which can affect system performance and response times.

Overfitting:

- **Challenge:** Models that are too complex might overfit the training data and fail to generalize well to new or unseen data.
- **Impact:** The recommendation system might perform well on historical data but poorly on new data or in real-world scenarios.

Lack of Contextual Understanding:

- **Challenge:** Basic recommendation systems may not account for contextual factors such as time, location, or mood.
- **Impact:** Recommendations might not be as relevant if they don't consider the user's current situation or preferences.

Bias and Diversity:

- **Challenge:** Recommendation algorithms might reinforce existing biases or fail to promote diverse content.
- **Impact:** Users might receive repetitive or homogeneous recommendations, which can limit exposure to varied content.

User Privacy Concerns:

- **Challenge:** Collecting and processing user data for recommendations can raise privacy issues and data security concerns.
- **Impact:** Users might be concerned about how their data is used, leading to trust and compliance issues.

Chapter-3

RESULTS AND DISCUSSIONS

1.1 Streamlit Interface:

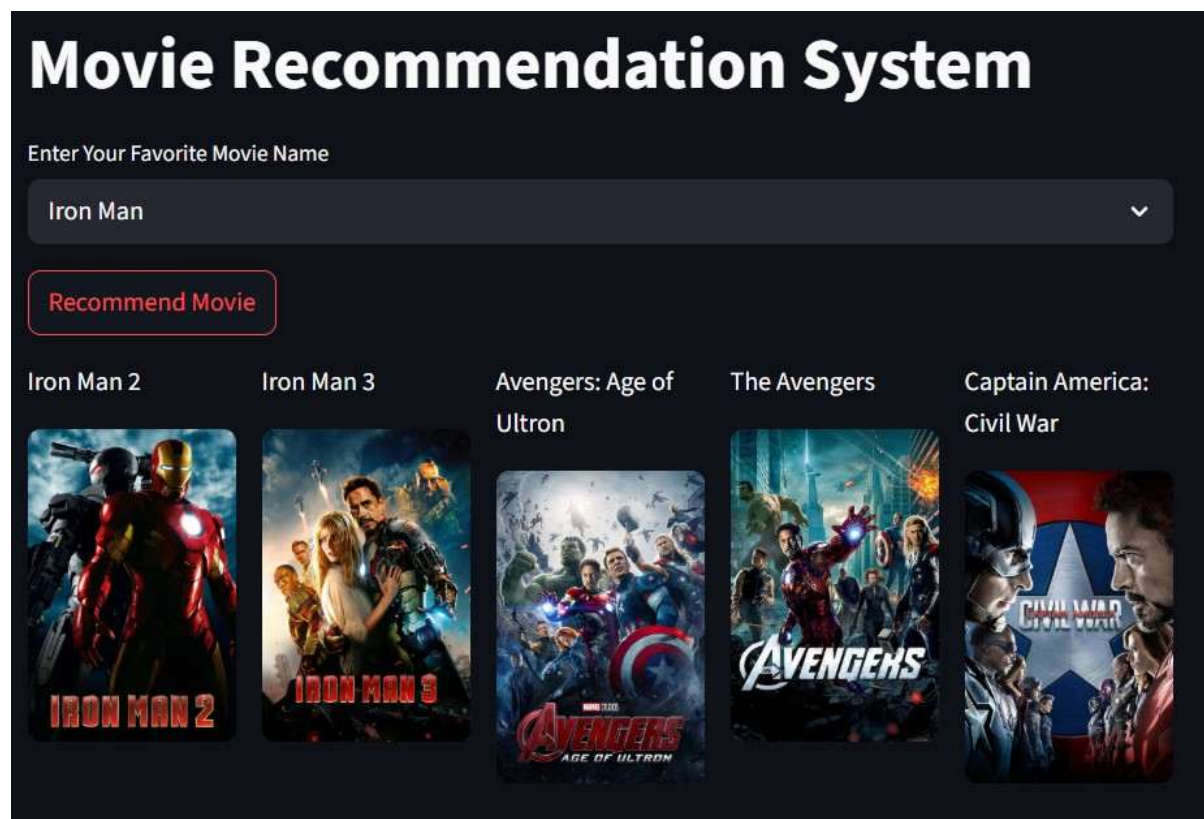
- **Select Box:** For users to select a movie from a list.
- **Recommendation Display:** To show recommended movies and their posters.
- **Error Handling:** Handle cases where no recommendations are available or the movie title is not found.

Interface



The screenshot displays the Streamlit interface for a 'Movie Recommendation System'. The title 'Movie Recommendation System' is prominently displayed at the top in a large, bold, white font. Below the title, there is a text input field with the placeholder 'Enter Your Favorite Movie Name'. The input field contains the text 'Avatar'. Below the input field, there is a button labeled 'Recommend Movie'. The entire interface is set against a dark background.

1.2 Result



1.3 Discussions

In addition to enhancing user engagement through personalized recommendations, our movie recommendation system project will focus on creating an intuitive user interface that allows for easy navigation and interaction. Key features like search filters and the ability to rate movies will improve user experience. Ethical considerations are paramount; we will prioritize data privacy by anonymizing user information and ensuring transparency about data usage. To address the diversity in user preferences, we may implement demographic-based filtering and clustering techniques to tailor recommendations to specific groups. Real-time data integration will enhance the relevance of suggestions by aligning them with current trends and events. Establishing feedback loops will allow users to provide input on their recommendations, enabling continuous refinement of our algorithms. As our user base grows, we will ensure technical scalability by leveraging cloud services and exploring distributed computing frameworks. Community engagement features, such as sharing favorite movies, will foster a sense of connection among users and provide additional data for refining recommendations. Finally, this project lays the groundwork for future research into advanced techniques like deep learning and natural language processing to further improve the recommendation process. Overall, by addressing these multifaceted aspects, we aim to create a robust system that significantly enhances the movie-watching experience.

Chapter-4

CONCLUSION AND FUTURE SCOPE

4.1 Conclusion

Movie Recommendation System- A Gateway to Personalized Movies.

The movie recommender system project effectively showcases the application of data processing and machine learning techniques to deliver personalized movie suggestions. By integrating movie metadata and credits, the system preprocesses and extracts relevant features such as genres, keywords, and cast details, creating a comprehensive representation of each movie. Utilizing cosine similarity, it identifies and recommends movies that are similar to the user's selection, providing relevant suggestions based on content characteristics. The web interface, developed with Streamlit, offers an intuitive and interactive experience, allowing users to easily receive recommendations. Despite its success, the system has limitations, including scalability concerns, a reliance on static data, and a lack of user feedback integration. Future enhancements could involve incorporating collaborative filtering, expanding feature sets, updating datasets dynamically, and optimizing performance to handle larger volumes of data. Overall, the project highlights the effectiveness of content based recommendation algorithms and sets a foundation for further advancements in personalized movie recommendations.

4.2 FUTURE SCOPE

Incorporation of Collaborative Filtering:

- **Hybrid Models:** Combining content-based and collaborative filtering approaches can enhance recommendation accuracy. By leveraging user interactions and preferences alongside content features, the system can offer more personalized suggestions.
- **User-Item Matrix:** Implementing a collaborative filtering model that uses a user item interaction matrix can help in recommending movies based on user behavior and preferences.

Real-time Data Updates:

- **Dynamic Data Integration:** Incorporating real-time data from movie databases or streaming platforms can keep the recommendations current and relevant. Automated data fetching and updating mechanisms can ensure the system adapts to new releases and changing user preferences.

Enhanced Feature Engineering:

- **Deep Learning Approaches:** Exploring deep learning models, such as neural networks, to extract more nuanced features from movie descriptions, reviews, and other sources can improve recommendation quality.
- **Sentiment Analysis:** Analyzing movie reviews and sentiments could provide additional insights into movie qualities and user preferences.

